CHAPTER 2

PostgreSQL

PostgreSQL is the hammer of the database world. It's commonly understood, often readily available, and sturdy, and it solves a surprising number of problems if you swing hard enough. No one can hope to be an expert builder without understanding this most common of tools.

PostgreSQL (or just "Postgres") is a relational database management system (or RDBMS for short). Relational databases are set-theory-based systems in which data is stored in two-dimensional tables consisting of data rows and strictly enforced column types. Despite the growing interest in newer database trends, the relational style remains the most popular and probably will for quite some time. Even in a book that focuses on non-relational "NoSQL" systems, a solid grounding in RDBMS remains essential. PostgreSQL is quite possibly the finest open source RDBMS available, and in this chapter you'll see that it provides a great introduction to this paradigm.

While the prevalence of relational databases can be partially attributed to their vast toolkits (triggers, stored procedures, views, advanced indexes), their data safety (via ACID compliance), or their mind share (many programmers speak and think relationally), query pliancy plays a central role as well. Unlike some other databases, you don't need to know in advance *how* you plan to use the data. If a relational schema is normalized, queries are flexible. You can start storing data and worrying about how exactly you'll use it later on, even changing your entire querying model over time as your needs change.

That's Post-greS-Q-L

PostgreSQL is by far the oldest and most battle-tested database in this book. It has domain-specific plug-ins for things like natural language parsing, multidimensional indexing, geographic queries, custom datatypes, and much more. It has sophisticated transaction handling and built-in stored procedures

So, What's with the Name?

PostgreSQL has existed in the current project incarnation since 1995, but its roots go back much further. The project was originally created at UC Berkeley in the early 1970s and called the Interactive Graphics and Retrieval System, or "Ingres" for short. In the 1980s, an improved version was launched post-Ingres—shortened to "Postgres." The project ended at Berkeley proper in 1993 but was picked up again by the open source community as Postgres95. It was later renamed to PostgreSQL in 1996 to denote its rather new SQL support and has retained the name ever since.

for a dozen languages, and it runs on a variety of platforms. PostgreSQL has built-in Unicode support, sequences, table inheritance, and subselects, and it is one of the most ANSI SQL-compliant relational databases on the market. It's fast and reliable, can handle terabytes of data, and has been proven to run in high-profile production projects such as Skype, Yahoo!, France's Caisse Nationale d'Allocations Familiales (CNAF), Brazil's Caixa Bank, and the United States' Federal Aviation Administration (FAA).

You can install PostgreSQL in many ways, depending on your operating system.[1] Once you have Postgres installed, create a schema called `7dbs` using the following command:

```
$ createdb 7dbs
```

We'll be using the `7dbs` schema for the remainder of this chapter.

Day 1: Relations, CRUD, and Joins

While we won't assume that you're a relational database expert, we do assume that you've confronted a database or two in the past. If so, odds are good that the database was relational. We'll start with creating our own schemas and populating them. Then we'll take a look at querying for values and finally explore what makes relational databases so special: the table join.

Like most databases you'll read about, Postgres provides a back-end server that does all of the work and a command-line shell to connect to the running server. The server communicates through port 5432 by default, which you can connect to the shell using the `psql` command. Let's connect to our `7dbs` schema now:

```
$ psql 7dbs
```

1. http://www.postgresql.org/download/

PostgreSQL prompts with the name of the database followed by a hash mark (#) if you run as an administrator and by a dollar sign ($) as a regular user. The shell also comes equipped with perhaps the best built-in documentation that you will find in any console. Typing \h lists information about SQL commands and \? helps with psql-specific commands, namely those that begin with a backslash. You can find usage details about each SQL command in the following way (the output that follows is truncated):

```
7dbs=# \h CREATE INDEX
Command:     CREATE INDEX
Description: define a new index
Syntax:
CREATE [ UNIQUE ] INDEX [ CONCURRENTLY ] [ name ] ON table [ USING method ]
    ( { column | ( expression ) } [ opclass ] [ ASC | DESC ] [ NULLS ...
    [ WITH ( storage_parameter = value [, ... ] ) ]
    [ TABLESPACE tablespace ]
    [ WHERE predicate ]
```

Before we dig too deeply into Postgres, it would be good to familiarize yourself with this useful tool. It's worth looking over (or brushing up on) a few common commands, such as SELECT and CREATE TABLE.

Starting with SQL

PostgreSQL follows the SQL convention of calling relations TABLEs, attributes COLUMNs, and tuples ROWs. For consistency, we will use this terminology, though you may occasionally encounter the mathematical terms *relations*, *attributes*, and *tuples*. For more on these concepts, see "Mathematical Relations" in the text on page 12.

Working with Tables

PostgreSQL, being of the relational style, is a design-first database. First you design the schema; then you enter data that conforms to the definition of that schema.

On CRUD

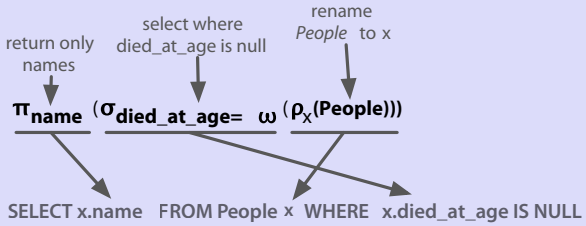
CRUD is a useful mnemonic for remembering the basic data management operations: *Create*, *Read*, *Update*, and *Delete*. These generally correspond to inserting new records (*creating*), modifying existing records (*updating*), and removing records you no longer need (*deleting*). All of the other operations you use a database for (any crazy query you can dream up) are *read operations*. If you can CRUD, you can do just about anything. We'll use this term throughout the book.

Mathematical Relations

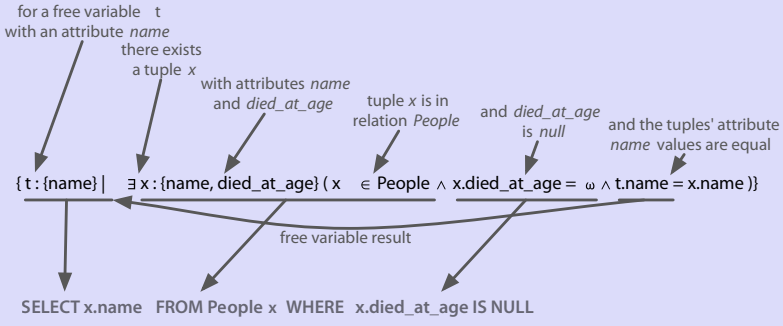
Relational databases are so named because they contain *relations* (tables). Tables are sets of *tuples* (rows) that map *attributes* to atomic values—for example, `{name: 'Genghis Khan', died_at_age: 65}`. The available attributes are defined by a *header*, which is a tuple of attributes mapped to some *domain* or constraining type (columns, for example `{name: string, age: int}`). That's the gist of the relational structure.

Implementations are much more practically minded than the names imply, despite sounding so mathematical. So why bring them up? We're trying to make the point that relational databases are *relational* based on mathematics. They aren't relational because tables "relate" to each other via foreign keys. Whether any such constraints exist is beside the point.

The power of the model is certainly in the math, even though the math is largely hidden from you. This magic allows users to express powerful queries while the system optimizes based on predefined patterns. RDBMSs are built atop a set theory branch called *relational algebra*—a combination of selections (`WHERE ...`), projections (`SELECT ...`), Cartesian products (`JOIN ...`), and more, as shown in the figure that follows.



You can imagine relations as arrays of arrays, where a table is an array of rows, each of which contains an array of attribute/value pairs. One way of working with tables at the code level would be to iterate across all rows in a table and then iterate across each attribute/value pair within the row. But let's be real: that sounds like a real chore. Fortunately, relational queries are much more declarative—and fun to work with—than that. They're derived from a branch of mathematics known as *tuple relational calculus*, which can be converted to relational algebra. PostgreSQL and other RDBMSs optimize queries by performing this conversion and simplifying the algebra. You can see that the SQL in the diagram that follows is the same as in the previous diagram.



Creating a table involves giving the table a name and a list of columns with types and (optional) constraint information. Each table should also nominate a unique identifier column to pinpoint specific rows. That identifier is called a PRIMARY KEY. The SQL to create a countries table looks like this:

```
CREATE TABLE countries (
  country_code char(2) PRIMARY KEY,
  country_name text UNIQUE
);
```

This new table will store a set of rows, where each is identified by a two-character code and the name is unique. These columns both have *constraints*. The PRIMARY KEY constrains the country_code column to disallow duplicate country codes. Only one us and one gb may exist. We explicitly gave country_name a similar unique constraint, although it is not a primary key. We can populate the countries table by inserting a few rows.

```
INSERT INTO countries (country_code, country_name)
VALUES ('us','United States'), ('mx','Mexico'), ('au','Australia'),
       ('gb','United Kingdom'), ('de','Germany'), ('ll','Loompaland');
```

Let's test our unique constraint. Attempting to add a duplicate country_name will cause our unique constraint to fail, thus disallowing insertion. Constraints are how relational databases such as PostgreSQL ensure kosher data.

```
INSERT INTO countries
VALUES ('uk','United Kingdom');
```

This will return an error indicating that the Key (country_name)=(United Kingdom) already exists.

We can validate that the proper rows were inserted by reading them using the SELECT...FROM table command.

```
SELECT *
FROM countries;
```

```
 country_code | country_name
--------------+---------------
 us           | United States
 mx           | Mexico
 au           | Australia
 gb           | United Kingdom
 de           | Germany
 ll           | Loompaland
(6 rows)
```

According to any respectable map, Loompaland isn't a real place, so let's remove it from the table. You specify which row to remove using the WHERE clause. The row whose country_code equals ll will be removed.

```
DELETE FROM countries
WHERE country_code = 'll';
```

With only real countries left in the countries table, let's add a cities table. To ensure any inserted country_code also exists in our countries table, we add the REFERENCES keyword. Because the country_code column references another table's key, it's known as the *foreign key* constraint.

```
CREATE TABLE cities (
  name text NOT NULL,
  postal_code varchar(9) CHECK (postal_code <> ''),
  country_code char(2) REFERENCES countries,
  PRIMARY KEY (country_code, postal_code)
);
```

This time, we constrained the name in cities by disallowing NULL values. We constrained postal_code by checking that no values are empty strings (<> means *not equal*). Furthermore, because a PRIMARY KEY uniquely identifies a row, we created a compound key: country_code + postal_code. Together, they uniquely define a row.

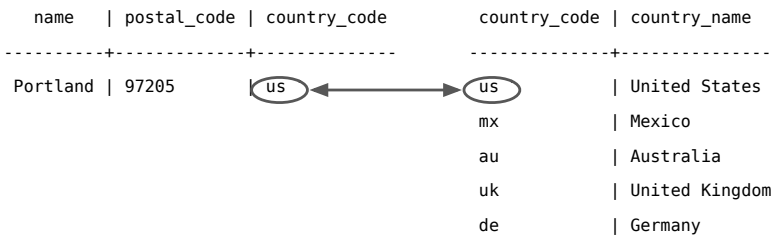
Postgres also has a rich set of datatypes, by far the richest amongst the databases in this book. You've just seen three different string representations: text (a string of any length), varchar(9) (a string of variable length up to nine characters), and char(2) (a string of exactly two characters). With our schema in place, let's insert *Toronto, CA*.

```
INSERT INTO cities
VALUES ('Toronto','M4C1B5','ca');
```

```
ERROR:  insert or update on table "cities" violates foreign key constraint
  "cities_country_code_fkey"
DETAIL:  Key (country_code)=(ca) is not present in table "countries".
```

This failure is good! Because country_code REFERENCES countries, the country_code must exist in the countries table. As shown in the figure on page 15, the REFERENCES keyword constrains fields to another table's primary key. This is called *maintaining referential integrity*, and it ensures our data is always correct.

It's worth noting that NULL is valid for cities.country_code because NULL represents the lack of a value. If you want to disallow a NULL country_code reference, you would define the table cities column like this: country_code char(2) REFERENCES countries NOT NULL.



Now let's try another insert, this time with a U.S. city (quite possibly the greatest of U.S. cities).

```
INSERT INTO cities
VALUES ('Portland','87200','us');
```

```
INSERT 0 1
```

This is a successful insert, to be sure. But we mistakenly entered a postal_code that doesn't actually exist in Portland. One postal code that does exist and may just belong to one of the authors is *97206*. Rather than delete and reinsert the value, we can update it inline.

```
UPDATE cities
SET postal_code = '97206'
WHERE name = 'Portland';
```

We have now Created, Read, Updated, and Deleted table rows.

Join Reads

All of the other databases you'll read about in this book perform CRUD operations as well. What sets relational databases like PostgreSQL apart is their ability to join tables together when reading them. Joining, in essence, is an operation taking two separate tables and combining them in some way to return a single table. It's somewhat like putting together puzzle pieces to create a bigger, more complete picture.

The basic form of a join is the *inner join*. In the simplest form, you specify two columns (one from each table) to match by, using the ON keyword.

```
SELECT cities.*, country_name
FROM cities INNER JOIN countries /* or just FROM cities JOIN countries */
  ON cities.country_code = countries.country_code;
```

```
 country_code |   name   | postal_code | country_name
--------------+----------+-------------+---------------
 us           | Portland | 97206       | United States
```

The join returns a single table, sharing all columns' values of the cities table plus the matching country_name value from the countries table.

You can also join a table like cities that has a compound primary key. To test a compound join, let's create a new table that stores a list of venues.

A venue exists in both a *postal code* and a specific *country*. The *foreign key* must be two columns that reference both cities *primary key* columns. (MATCH FULL is a constraint that ensures either both values exist or both are NULL.)

```
CREATE TABLE venues (
  venue_id SERIAL PRIMARY KEY,
  name varchar(255),
  street_address text,
  type char(7) CHECK ( type in ('public','private') ) DEFAULT 'public',
  postal_code varchar(9),
  country_code char(2),
  FOREIGN KEY (country_code, postal_code)
    REFERENCES cities (country_code, postal_code) MATCH FULL
);
```

This venue_id column is a common primary key setup: automatically incremented integers (1, 2, 3, 4, and so on). You make this identifier using the SERIAL keyword. (MySQL has a similar construct called AUTO_INCREMENT.)

```
INSERT INTO venues (name, postal_code, country_code)
VALUES ('Crystal Ballroom', '97206', 'us');
```

Although we did not set a venue_id value, creating the row populated it.

Back to our compound join. Joining the venues table with the cities table requires *both* foreign key columns. To save on typing, you can alias the table names by following the real table name directly with an alias, with an optional AS between (for example, venues v or venues AS v).

```
SELECT v.venue_id, v.name, c.name
FROM venues v INNER JOIN cities c
  ON v.postal_code=c.postal_code AND v.country_code=c.country_code;
```

```
 venue_id |       name       |   name
----------+------------------+----------
        1 | Crystal Ballroom | Portland
```

You can optionally request that PostgreSQL return columns after insertion by ending the query with a RETURNING statement.

```
INSERT INTO venues (name, postal_code, country_code)
VALUES ('Voodoo Doughnut', '97206', 'us') RETURNING venue_id;
```

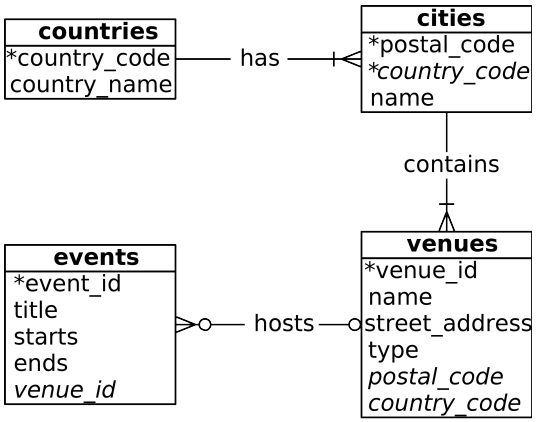
```
 id
- - - -
  2
```

This provides the new venue_id without issuing another query.

The Outer Limits

In addition to inner joins, PostgreSQL can also perform *outer joins*. Outer joins are a way of merging two tables when the results of one table must always be returned, whether or not any matching column values exist on the other table.

It's easiest to give an example, but to do that, we'll create a new table named events. This one is up to you. Your events table should have these columns: a SERIAL integer event_id, a title, starts and ends (of type *timestamp*), and a venue_id (foreign key that references venues). A schema definition diagram covering all the tables we've made so far is shown in the following figure.



After creating the events table, INSERT the following values (timestamps are inserted as a string like *2018-02-15 17:30*) for two holidays and a club we *do not talk about*:

title	starts	ends	venue_id	event_id
Fight Club	2018-02-15 17:30:00	2018-02-15 19:30:00	2	1
April Fools Day	2018-04-01 00:00:00	2018-04-01 23:59:00		2
Christmas Day	2018-02-15 19:30:00	2018-12-25 23:59:00		3

Let's first craft a query that returns an event title and venue name as an inner join (the word INNER from INNER JOIN is not required, so leave it off here).

```
SELECT e.title, v.name
FROM events e JOIN venues v
  ON e.venue_id = v.venue_id;
```

```
       title  |       name
--------------+------------------
 Fight Club   | Voodoo Doughnut
```

INNER JOIN will return a row only *if the column values match*. Because we can't have NULL venues.venue_id, the two NULL events.venue_ids refer to nothing. Retrieving

all of the events, whether or not they have a venue, requires a LEFT OUTER JOIN (shortened to LEFT JOIN).

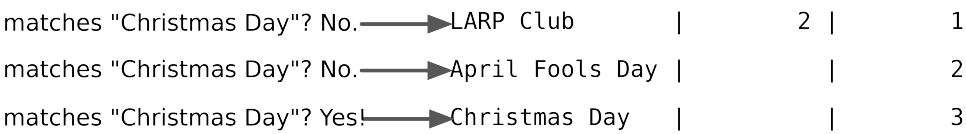
```
SELECT e.title, v.name
FROM events e LEFT JOIN venues v
ON e.venue_id = v.venue_id;
```

```
      title       |      name
------------------+----------------
 Fight Club       | Voodoo Doughnut
 April Fools Day |
 Christmas Day    |
```

If you require the inverse, all venues and only matching events, use a RIGHT JOIN. Finally, there's the FULL JOIN, which is the union of LEFT and RIGHT; you're guaranteed all values from each table, joined wherever columns match.

Fast Lookups with Indexing

The speed of PostgreSQL (and any other RDBMS) lies in its efficient management of blocks of data, reduced disk reads, query optimization, and other techniques. But those only go so far in fetching results quickly. If we select the title of *Christmas Day* from the events table, the algorithm must *scan* every row for a match to return. Without an *index*, each row must be read from disk to know whether a query should return it. See the following.



An index is a special data structure built to avoid a full table scan when performing a query. When running CREATE TABLE commands, you may have noticed a message like this:

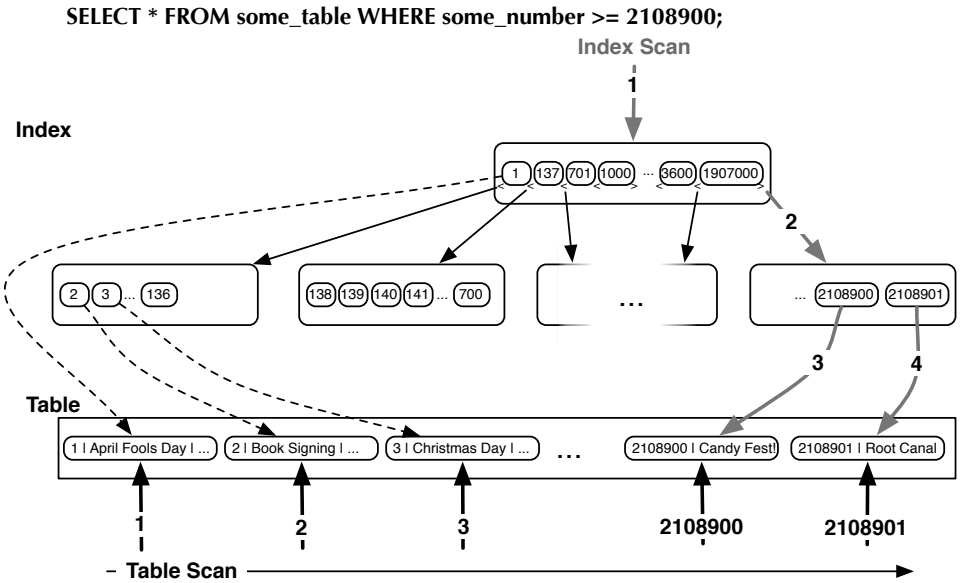
```
CREATE TABLE / PRIMARY KEY will create implicit index "events_pkey" \
for table "events"
```

PostgreSQL automatically creates an index on the primary key—in particular a B-tree index—where the key is the primary key value and where the value points to a row on disk, as shown in the top figure on page 19. Using the UNIQUE keyword is another way to force an index on a table column.

You can explicitly add a hash index using the CREATE INDEX command, where each value must be unique (like a hashtable or a map).

```
CREATE INDEX events_title
  ON events USING hash (title);
```

For less-than/greater-than/equals-to matches, we want an index more flexible than a simple hash, like a B-tree, which can match on ranged queries (see the following figure).



Consider a query to find all events that are on or after April 1.

```
SELECT *
FROM events
WHERE starts >= '2018-04-01';
```

```
 event_id |      title       |       starts        | ...
----------+------------------+---------------------+-----
        2 | April Fools Day | 2018-04-01 00:00:00 | ...
        3 | Christmas Day    | 2018-12-25 00:00:00 | ...
```

For this, a tree is the perfect data structure. To index the starts column with a B-tree, use this:

```
CREATE INDEX events_starts
  ON events USING btree (starts);
```

Now our query over a range of dates will avoid a full table scan. It makes a huge difference when scanning millions or billions of rows.

We can inspect our work with this command to list all indexes in the schema:

```
7dbs=# \di
```

It's worth noting that when you set a FOREIGN KEY constraint, PostgreSQL will *not* automatically create an index on the targeted column(s). You'll need to create an index on the targeted column(s) yourself. Even if you don't like using database constraints (that's right, we're looking at you, Ruby on Rails developers), you will often find yourself creating indexes on columns you plan to join against in order to help speed up foreign key joins.

Day 1 Wrap-Up

We sped through a lot today and covered many terms. Here's a recap:

Term	Definition
Column	A domain of values of a certain type, sometimes called an *attribute*
Row	An object comprised of a set of column values, sometimes called a *tuple*
Table	A set of rows with the same columns, sometimes called a *relation*
Primary key	The unique value that pinpoints a specific row
Foreign key	A data constraint that ensures that each entry in a column in one table uniquely corresponds to a row in another table (or even the same table)
CRUD	Create, Read, Update, Delete
SQL	Structured Query Language, the *lingua franca* of a relational database
Join	Combining two tables into one by some matching columns
Left join	Combining two tables into one by some matching columns or NULL if nothing matches the left table
Index	A data structure to optimize selection of a specific set of columns

Term	Definition
B-tree index	A good standard index; values are stored as a balanced tree data structure; very flexible; B-tree indexes are the default in Postgres
Hash index	Another good standard index in which each index value is unique; hash indexes tend to offer better performance for comparison operations than B-tree indexes but are less flexible and don't allow for things like range queries

Relational databases have been the *de facto* data management strategy for forty years—many of us began our careers in the midst of their evolution. Others may disagree, but we think that understanding "NoSQL" databases is a non-starter without rooting ourselves in this paradigm, even if for just a brief sojourn. So we looked at some of the core concepts of the relational model via basic SQL queries and undertook a light foray into some mathematical foundations. We will expound on these root concepts tomorrow.

Day 1 Homework

Find

1. Find the PostgreSQL documentation online and bookmark it.
2. Acquaint yourself with the command-line `\?` and `\h` output.
3. We briefly mentioned the `MATCH FULL` constraint. Find some information on the other available types of `MATCH` constraints.

Do

1. Select all the tables we created (and only those) from `pg_class` and examine the table to get a sense of what kinds of metadata Postgres stores about tables.
2. Write a query that finds the country name of the Fight Club event.
3. Alter the `venues` table such that it contains a Boolean column called `active` with a default value of `TRUE`.

Day 2: Advanced Queries, Code, and Rules

Yesterday we saw how to define tables, populate them with data, update and delete rows, and perform basic reads. Today we'll dig even deeper into the myriad ways that PostgreSQL can query data. We'll see how to group similar values, execute code on the server, and create custom interfaces using *views*

and *rules*. We'll finish the day by using one of PostgreSQL's contributed packages to flip tables on their heads.

Aggregate Functions

An aggregate query groups results from several rows by some common criteria. It can be as simple as counting the number of rows in a table or calculating the average of some numerical column. They're powerful SQL tools and also a lot of fun.

Let's try some aggregate functions, but first we'll need some more data in our database. Enter your own country into the countries table, your own city into the cities table, and your own address as a venue (which we just named *My Place*). Then add a few records to the events table.

Here's a quick SQL tip: Rather than setting the venue_id explicitly, you can sub-SELECT it using a more human-readable title. If *Moby* is playing at the *Crystal Ballroom*, set the venue_id like this:

```
INSERT INTO events (title, starts, ends, venue_id)
  VALUES ('Moby', '2018-02-06 21:00', '2018-02-06 23:00', (
    SELECT venue_id
    FROM venues
    WHERE name = 'Crystal Ballroom'
  )
);
```

Populate your events table with the following data (to enter *Valentine's Day* in PostgreSQL, you can escape the apostrophe with two, such as Heaven''s Gate):

```
       title     |       starts        |        ends         |      venue
-----------------+---------------------+---------------------+---------------
 Wedding         | 2018-02-26 21:00:00 | 2018-02-26 23:00:00 | Voodoo Doughnut
 Dinner with Mom | 2018-02-26 18:00:00 | 2018-02-26 20:30:00 | My Place
 Valentine's Day | 2018-02-14 00:00:00 | 2018-02-14 23:59:00 |
```

With our data set up, let's try some aggregate queries. The simplest aggregate function is count(), which is fairly self-explanatory. Counting all titles that contain the word *Day* (note: % is a wildcard on LIKE searches), you should receive a value of 3.

```
SELECT count(title)
FROM events
WHERE title LIKE '%Day%';
```

To get the first start time and last end time of all events at the Crystal Ballroom, use min() (return the smallest value) and max() (return the largest value).

```
SELECT min(starts), max(ends)
FROM events INNER JOIN venues
  ON events.venue_id = venues.venue_id
WHERE venues.name = 'Crystal Ballroom';
```

```
         min         |         max
---------------------+---------------------
 2018-02-06 21:00:00 | 2018-02-06 23:00:00
```

Aggregate functions are useful but limited on their own. If we wanted to count all events at each venue, we could write the following for each venue ID:

```
SELECT count(*) FROM events WHERE venue_id = 1;
SELECT count(*) FROM events WHERE venue_id = 2;
SELECT count(*) FROM events WHERE venue_id = 3;
SELECT count(*) FROM events WHERE venue_id IS NULL;
```

This would be tedious (intractable even) as the number of venues grows. This is where the GROUP BY command comes in handy.

Grouping

GROUP BY is a shortcut for running the previous queries all at once. With GROUP BY, you tell Postgres to place the rows into groups and then perform some aggregate function (such as count()) on those groups.

```
SELECT venue_id, count(*)
FROM events
GROUP BY venue_id;
```

```
 venue_id | count
----------+-------
        1 |     1
        2 |     2
        3 |     1
        4 |     3
```

It's a nice list, but can we filter by the count() function? Absolutely. The GROUP BY condition has its own filter keyword: HAVING. HAVING is like the WHERE clause, except it can filter by aggregate functions (whereas WHERE cannot).

The following query SELECTs the most popular venues, those with two or more events:

```
SELECT venue_id
FROM events
GROUP BY venue_id
HAVING count(*) >= 2 AND venue_id IS NOT NULL;
```

```
 venue_id | count
----------+-------
        2 |     2
```

You can use GROUP BY without any aggregate functions. If you call SELECT... FROM...GROUP BY on one column, you get only unique values.

```
SELECT venue_id FROM events GROUP BY venue_id;
```

This kind of grouping is so common that SQL has a shortcut for it in the DISTINCT keyword.

```
SELECT DISTINCT venue_id FROM events;
```

The results of both queries will be identical.

GROUP BY in MySQL

If you tried to run a SELECT statement with columns not defined under a GROUP BY in MySQL, you would be shocked to see that it works. This originally made us question the necessity of window functions. But when we inspected the data MySQL returns more closely, we found it will return only a random row of data along with the count, not all relevant results. Generally, that's not useful (and quite potentially dangerous).

Window Functions

If you've done any sort of production work with a relational database in the past, you are likely familiar with aggregate queries. They are a common SQL staple. *Window functions*, on the other hand, are not quite so common (PostgreSQL is one of the few open source databases to implement them).

Window functions are similar to GROUP BY queries in that they allow you to run aggregate functions across multiple rows. The difference is that they allow you to use built-in aggregate functions without requiring every single field to be grouped to a single row.

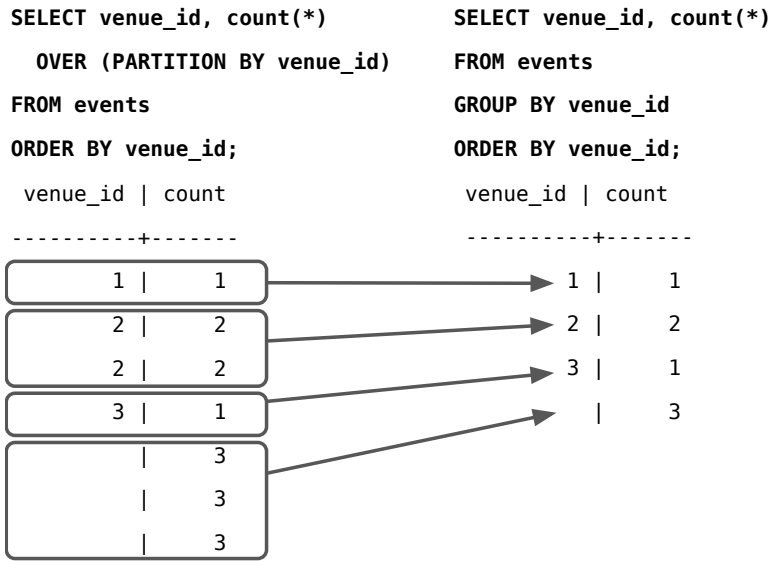
If we attempt to select the title column without grouping by it, we can expect an error.

```
SELECT title, venue_id, count(*)
FROM events
GROUP BY venue_id;
```

```
ERROR:  column "events.title" must appear in the GROUP BY clause or \
        be used in an aggregate function
```

We are counting up the rows by venue_id, and in the case of *Fight Club* and *Wedding*, we have two titles for a single venue_id. Postgres doesn't know *which* title to display.

Whereas a GROUP BY clause will return one record per matching group value, a window function, which does not collapse the results per group, can return a separate record for each row. For a visual representation, see the following figure.



Let's see an example of the sweet spot that window functions attempt to hit.

Window functions return all matches and replicate the results of any aggregate function.

```
SELECT title, count(*) OVER (PARTITION BY venue_id) FROM events;
```

```
    title     | count
--------------+-------
 Moby         |     1
 Fight Club   |     1
 House Party  |     3
 House Party  |     3
 House Party  |     3
(5 rows)
```

We like to think of PARTITION BY as akin to GROUP BY, but rather than grouping the results outside of the SELECT attribute list (and thus combining the results into fewer rows), it returns grouped values as any other field (calculating on the grouped variable but otherwise just another attribute). Or in SQL parlance, it returns the results of an aggregate function OVER a PARTITION of the result set.

Transactions

Transactions are the bulwark of relational database consistency. *All or nothing*, that's the transaction motto. Transactions ensure that every command of a set is executed. If anything fails along the way, all of the commands are rolled back as if they never happened.

PostgreSQL transactions follow ACID compliance, which stands for:

- Atomic (either all operations succeed or none do)
- Consistent (the data will always be in a good state and never in an inconsistent state)
- Isolated (transactions don't interfere with one another)
- Durable (a committed transaction is safe, even after a server crash)

We should note here that *consistency* in ACID is different from *consistency* in CAP (covered in Appendix 2, *The CAP Theorem*, on page 315).

Unavoidable Transactions

Up until now, every command we've executed in psql has been implicitly wrapped in a transaction. If you executed a command, such as DELETE FROM account WHERE total < 20; and the database crashed halfway through the delete, you wouldn't be stuck with half a table. When you restart the database server, that command will be rolled back.

We can wrap any transaction within a BEGIN TRANSACTION block. To verify atomicity, we'll kill the transaction with the ROLLBACK command.

```
BEGIN TRANSACTION;
  DELETE FROM events;
ROLLBACK;
SELECT * FROM events;
```

The events all remain. Transactions are useful when you're modifying two tables that you don't want out of sync. The classic example is a debit/credit system for a bank, where money is moved from one account to another:

```
BEGIN TRANSACTION;
  UPDATE account SET total=total+5000.0 WHERE account_id=1337;
  UPDATE account SET total=total-5000.0 WHERE account_id=45887;
END;
```

If something happened between the two updates, this bank just lost five grand. But when wrapped in a transaction block, the initial update is rolled back, even if the server explodes.

Stored Procedures

Every command we've seen until now has been declarative in the sense that we've been able to get our desired result set using just SQL (which is quite powerful in itself). But sometimes the database doesn't give us everything we need natively and we need to run some code to fill in the gaps. At that point, though, you need to decide *where* the code is going to run. Should it run *in* Postgres or should it run on the application side?

If you decide you want the database to do the heavy lifting, Postgres offers *stored procedures*. Stored procedures are extremely powerful and can be used to do an enormous range of tasks, from performing complex mathematical operations that aren't supported in SQL to triggering cascading series of events to pre-validating data before it's written to tables and far beyond. On the one hand, stored procedures can offer huge performance advantages. But the architectural costs can be high (and sometimes not worth it). You may avoid streaming thousands of rows to a client application, but you have also bound your application code to this database. And so the decision to use stored procedures should not be made lightly.

Caveats aside, let's create a procedure (or `FUNCTION`) that simplifies `INSERT`ing a new event at a venue without needing the `venue_id`. Here's what the procedure will accomplish: if the venue doesn't exist, it will be created first and then referenced in the new event. The procedure will also return a Boolean indicating whether a new venue was added as a helpful bonus.

What About Vendor Lock-in?

When relational databases hit their heyday, they were the Swiss Army knife of technologies. You could store nearly anything—even programming entire projects in them (for example, Microsoft Access). The few companies that provided this software promoted use of proprietary differences and then took advantage of this corporate reliance by charging enormous license and consulting fees. This was the dreaded *vendor lock-in* that newer programming methodologies tried to mitigate in the 1990s and early 2000s.

The zeal to neuter the vendors, however, birthed maxims such as *no logic in the database*. This is a shame because relational databases are capable of so many varied data management options. Vendor lock-in has not disappeared. Many actions we investigate in this book are highly implementation-specific. However, it's worth knowing how to use databases to their fullest extent before deciding to skip tools such as stored procedures solely because they're implementation-specific.

postgres/add_event.sql

```
CREATE OR REPLACE FUNCTION add_event(
  title text,
  starts timestamp,
  ends timestamp,
  venue text,
  postal varchar(9),
  country char(2))
RETURNS boolean AS $$
DECLARE
  did_insert boolean := false;
  found_count integer;
  the_venue_id integer;
BEGIN
  SELECT venue_id INTO the_venue_id
  FROM venues v
  WHERE v.postal_code=postal AND v.country_code=country AND v.name ILIKE venue
  LIMIT 1;

  IF the_venue_id IS NULL THEN
    INSERT INTO venues (name, postal_code, country_code)
    VALUES (venue, postal, country)
    RETURNING venue_id INTO the_venue_id;

    did_insert := true;
  END IF;

  -- Note: this is a notice, not an error as in some programming languages
  RAISE NOTICE 'Venue found %', the_venue_id;

  INSERT INTO events (title, starts, ends, venue_id)
  VALUES (title, starts, ends, the_venue_id);

  RETURN did_insert;
END;
$$ LANGUAGE plpgsql;
```

You can import this external file into the current schema using the following command-line argument (if you don't feel like typing all that code).

```
7dbs=# \i add_event.sql
```

This stored procedure is run as a SELECT statement.

```
SELECT add_event('House Party', '2018-05-03 23:00',
  '2018-05-04 02:00', 'Run''s House', '97206', 'us');
```

Running it should return t (true) because this is the first use of the venue *Run's House*. This saves a client two round-trip SQL commands to the database (a SELECT and then an INSERT) and instead performs only one.

The language we used in the procedure we wrote is PL/pgSQL (which stands for Procedural Language/PostgreSQL). Covering the details of an entire

Choosing to Execute Database Code

This is the first of a number of places where you'll see this theme in this book: Does the code belong in your application or in the database? It's often a difficult decision, one that you'll have to answer uniquely for every application.

In many cases, you'll improve performance by as much as an order of magnitude. For example, you might have a complex application-specific calculation that requires custom code. If the calculation involves many rows, a stored procedure will save you from moving thousands of rows instead of a single result. The cost is splitting your application, your code, and your tests across two different programming paradigms.

programming language is beyond the scope of this book, but you can read much more about it in the online PostgreSQL documentation.[2]

In addition to PL/pgSQL, Postgres supports three more core languages for writing procedures: Tcl (PL/Tcl), Perl (PL/Perl), and Python (PL/Python). People have written extensions for a dozen more, including Ruby, Java, PHP, Scheme, and others listed in the public documentation. Try this shell command:

```
$ createlang 7dbs --list
```

It will list the languages installed in your database. The createlang command is also used to add new languages, which you can find online.[3]

Pull the Triggers

Triggers automatically fire stored procedures when some event happens, such as an insert or update. They allow the database to enforce some required behavior in response to changing data.

Let's create a new PL/pgSQL function that logs whenever an event is updated (we want to be sure no one changes an event and tries to deny it later). First, create a logs table to store event changes. A primary key isn't necessary here because it's just a log.

```
CREATE TABLE logs (
  event_id integer,
  old_title varchar(255),
  old_starts timestamp,
  old_ends timestamp,
  logged_at timestamp DEFAULT current_timestamp
);
```

2. http://www.postgresql.org/docs/9.0/static/plpgsql.html
3. http://www.postgresql.org/docs/9.0/static/app-createlang.html

Next, we build a function to insert old data into the log. The OLD variable represents the row about to be changed (NEW represents an incoming row, which we'll see in action soon enough). Output a notice to the console with the event_id before returning.

postgres/log_event.sql

```
CREATE OR REPLACE FUNCTION log_event() RETURNS trigger AS $$
DECLARE
BEGIN
  INSERT INTO logs (event_id, old_title, old_starts, old_ends)
  VALUES (OLD.event_id, OLD.title, OLD.starts, OLD.ends);
  RAISE NOTICE 'Someone just changed event #%', OLD.event_id;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Finally, we create our trigger to log changes after any row is updated.

```
CREATE TRIGGER log_events
  AFTER UPDATE ON events
  FOR EACH ROW EXECUTE PROCEDURE log_event();
```

So, it turns out our party at Run's House has to end earlier than we hoped. Let's change the event.

```
UPDATE events
SET ends='2018-05-04 01:00:00'
WHERE title='House Party';
```

```
NOTICE:  Someone just changed event #9
```

And the old end time was logged.

```
SELECT event_id, old_title, old_ends, logged_at
FROM logs;
```

```
event_id |  old_title  |      old_ends       |        logged_at
---------+-------------+---------------------+-------------------------
       9 | House Party | 2018-05-04 02:00:00 | 2017-02-26 15:50:31.939
```

Triggers can also be created before updates and before or after inserts.[4]

Viewing the World

Wouldn't it be great if you could use the results of a complex query just like any other table? Well, that's exactly what VIEWs are for. Unlike stored procedures, these aren't functions being executed but rather aliased queries. Let's say that we wanted to see only holidays that contain the word *Day* and have no venue. We could create a VIEW for that like this:

4. http://www.postgresql.org/docs/9.0/static/triggers.html

postgres/holiday_view_1.sql
```
CREATE VIEW holidays AS
  SELECT event_id AS holiday_id, title AS name, starts AS date
  FROM events
  WHERE title LIKE '%Day%' AND venue_id IS NULL;
```

Creating a view really is as simple as writing a query and prefixing it with CREATE VIEW some_view_name AS. Now you can query holidays like any other table. Under the covers it's the plain old events table. As proof, add *Valentine's Day* on *2018-02-14* to events and query the holidays view.

```
SELECT name, to_char(date, 'Month DD, YYYY') AS date
FROM holidays
WHERE date <= '2018-04-01';
```

```
       name       |        date
------------------+--------------------
 April Fools Day | April     01, 2018
 Valentine's Day  | February  14, 2018
```

Views are powerful tools for opening up complex queried data in a simple way. The query may be a roiling sea of complexity underneath, but all you see is a table.

If you want to add a new column to the holidays view, it will have to come from the underlying table. Let's alter the events table to have an array of associated colors.

```
ALTER TABLE events
ADD colors text ARRAY;
```

Because holidays are to have colors associated with them, let's update the VIEW query to contain the colors array.

```
CREATE OR REPLACE VIEW holidays AS
  SELECT event_id AS holiday_id, title AS name, starts AS date, colors
  FROM events
  WHERE title LIKE '%Day%' AND venue_id IS NULL;
```

Now it's a matter of setting an array or color strings to the holiday of choice. Unfortunately, we cannot update a VIEW directly.

```
UPDATE holidays SET colors = '{"red","green"}' where name = 'Christmas Day';
```

```
ERROR:  cannot update a view
HINT:  You need an unconditional ON UPDATE DO INSTEAD rule.
```

Looks like we need a RULE instead of a view.

Storing Views on Disk with Materialized Views

Though VIEWs like the holidays view mentioned previously are a convenient abstraction, they don't yield any performance gains over the SELECT queries that they alias. If you want VIEWs that *do* offer such gains, you should consider creating *materialized views*, which are different because they're stored on disk in a "real" table and thus yield performance gains because they restrict the number of tables that must be accessed to exactly one.

You can create materialized views just like ordinary views, except with a CREATE MATERIALIZED VIEW rather than CREATE VIEW statement. Materialized view tables are populated whenever you run the REFRESH command for them, which you can automate to run at defined intervals or in response to triggers. You can also create indexes on materialized views the same way that you can on regular tables.

The downside of materialized views is that they do increase disk space usage. But in many cases, the performance gains are worth it. In general, the more complex the query and the more tables it spans, the more performance gains you're likely to get vis-à-vis plain old SELECT queries or VIEWs.

What RULEs the School?

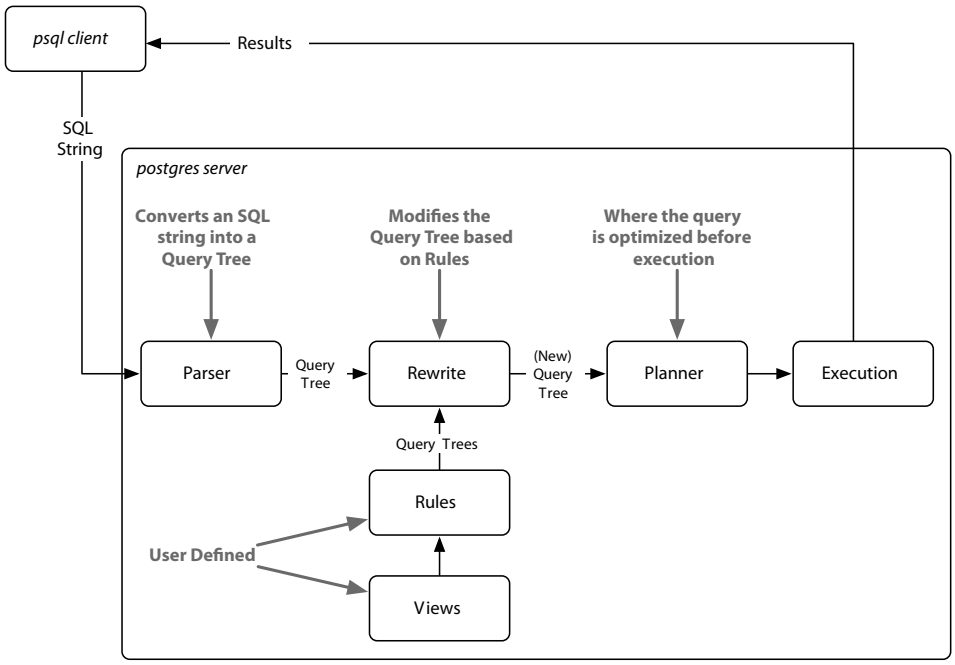
A RULE is a description of how to alter the parsed *query tree*. Every time Postgres runs an SQL statement, it parses the statement into a query tree (generally called an *abstract syntax tree*).

Operators and values become branches and leaves in the tree, and the tree is walked, pruned, and in other ways edited before execution. This tree is optionally rewritten by Postgres rules, before being sent on to the query planner (which also rewrites the tree to run optimally), and sends this final command to be executed. See how SQL gets executed in PostgreSQL in the figure on page 33.

In fact, a VIEW such as holidays *is* a RULE. We can prove this by taking a look at the execution plan of the holidays view using the EXPLAIN command (notice *Filter* is the WHERE clause, and *Output* is the column list).

```
EXPLAIN VERBOSE
  SELECT *
  FROM holidays;

                                    QUERY PLAN
-------------------------------------------------------------------------------
 Seq Scan on public.events  (cost=0.00..1.01 rows=1 width=44)
   Output: events.event_id, events.title, events.starts, events.colors
   Filter: ((events.venue_id IS NULL) AND
     ((events.title)::text ~~ '%Day%'::text))
```



Compare that to running EXPLAIN VERBOSE on the query from which we built the holidays VIEW. They're functionally identical.

```
EXPLAIN VERBOSE
  SELECT event_id AS holiday_id,
    title AS name, starts AS date, colors
  FROM events
  WHERE title LIKE '%Day%' AND venue_id IS NULL;

                                  QUERY PLAN
------------------------------------------------------------------------------
 Seq Scan on public.events  (cost=0.00..1.04 rows=1 width=57)
   Output: event_id, title, starts, colors
   Filter: ((events.venue_id IS NULL) AND
     ((events.title)::text ~~ '%Day%'::text))
```

So, to allow updates against our holidays view, we need to craft a RULE that tells Postgres what to do with an UPDATE. Our rule will capture updates to the holidays view and instead run the update on events, pulling values from the pseudorelations NEW and OLD. NEW functionally acts as the relation containing the values we're setting, while OLD contains the values we query by.

postgres/create_rule.sql
```
CREATE RULE update_holidays AS ON UPDATE TO holidays DO INSTEAD
  UPDATE events
  SET title = NEW.name,
      starts = NEW.date,
      colors = NEW.colors
  WHERE title = OLD.name;
```

With this rule in place, now we can update holidays directly.

```
UPDATE holidays SET colors = '{"red","green"}' where name = 'Christmas Day';
```

Next, let's insert *New Years Day* on *2013-01-01* into holidays. As expected, we need a rule for that too. No problem.

```
CREATE RULE insert_holidays AS ON INSERT TO holidays DO INSTEAD
  INSERT INTO ...
```

We're going to move on from here, but if you'd like to play more with RULEs, try to add a DELETE RULE.

I'll Meet You at the Crosstab

For our last exercise of the day, we're going to build a monthly calendar of events, where each month in the calendar year counts the number of events in that month. This kind of operation is commonly done by a *pivot table*. These constructs "pivot" grouped data around some other output, in our case a list of months. We'll build our pivot table using the crosstab() function.

Start by crafting a query to count the number of events per month each year. PostgreSQL provides an extract() function that returns some subfield from a date or timestamp, which aids in our grouping.

```
SELECT extract(year from starts) as year,
  extract(month from starts) as month, count(*)
FROM events
GROUP BY year, month
ORDER BY year, month;
```

To use crosstab(), the query must return three columns: rowid, category, and value. We'll be using the year as an ID, which means the other fields are category (the month) and value (the count).

The crosstab() function needs another set of values to represent months. This is how the function knows how many columns we need. These are the values that become the columns (the table to *pivot* against). So let's create a table to store a list of numbers. Because we'll only need the table for a few operations, we'll create an ephemeral table, which lasts only as long as the current Postgres session, using the CREATE TEMPORARY TABLE command.

```
CREATE TEMPORARY TABLE month_count(month INT);
INSERT INTO month_count VALUES (1),(2),(3),(4),(5),
  (6),(7),(8),(9),(10),(11),(12);
```

Now we're ready to call crosstab() with our two queries.

```
SELECT * FROM crosstab(
  'SELECT extract(year from starts) as year,
     extract(month from starts) as month, count(*)
   FROM events
   GROUP BY year, month
   ORDER BY year, month',
  'SELECT * FROM month_count'
);
```

```
ERROR:  a column definition list is required for functions returning "record"
```

Oops. An error occurred. This cryptic error is basically saying that the function is returning a set of records (rows) but it doesn't know how to label them. In fact, it doesn't even know what datatypes they are.

Remember, the pivot table is using our months as categories, but those months are just integers. So, we define them like this:

```
SELECT * FROM crosstab(
  'SELECT extract(year from starts) as year,
     extract(month from starts) as month, count(*)
   FROM events
   GROUP BY year, month
   ORDER BY year, month',
  'SELECT * FROM month_count'
) AS (
  year int,
  jan int, feb int, mar int, apr int, may int, jun int,
  jul int, aug int, sep int, oct int, nov int, dec int
) ORDER BY YEAR;
```

We have one column year (which is the row ID) and twelve more columns representing the months.

```
year | jan | feb | mar | apr | may | jun | jul | aug | sep | oct | nov | dec
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
2018 |     |   5 |     |   1 |   1 |     |     |     |     |     |     |   1
```

Go ahead and add a couple more events on another year just to see next year's event counts. Run the crosstab() function again, and enjoy the calendar.

Day 2 Wrap-Up

Today finalized the basics of PostgreSQL. What we're starting to see is that Postgres is more than just a server for storing vanilla datatypes and querying

them. Instead, it's a powerful data management engine that can reformat output data, store weird datatypes such as arrays, execute logic, and provide enough power to rewrite incoming queries.

Day 2 Homework

Find

1. Find the list of aggregate functions in the PostgreSQL docs.
2. Find a GUI program to interact with PostgreSQL, such as pgAdmin, Datagrip, or Navicat.

Do

1. Create a rule that captures `DELETE`s on venues and instead sets the active flag (created in the Day 1 homework) to `FALSE`.
2. A temporary table was not the best way to implement our event calendar pivot table. The `generate_series(a, b)` function returns a set of records, from a to b. Replace the `month_count` table `SELECT` with this.
3. Build a pivot table that displays every day in a single month, where each week of the month is a row and each day name forms a column across the top (seven days, starting with `Sunday` and ending with `Saturday`) like a standard month calendar. Each day should contain a count of the number of events for that date or should remain blank if no event occurs.

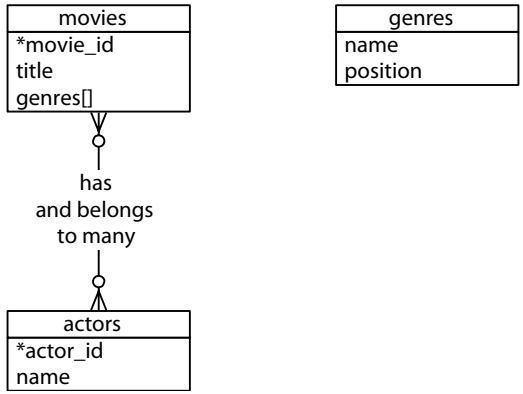
Day 3: Full Text and Multidimensions

We'll spend Day 3 investigating the many tools at our disposal to build a movie query system. We'll begin with the many ways PostgreSQL can search actor/movie names using fuzzy string matching. Then we'll discover the `cube` package by creating a movie suggestion system based on similar genres of movies we already like. Because these are all contributed packages, the implementations are special to PostgreSQL and not part of the SQL standard.

Often, when designing a relational database schema, you'll start with an entity diagram. We'll be writing a personal movie suggestion system that keeps track of movies, their genres, and their actors, as modeled in the figure on page 37.

Before we begin the Day 3 exercises, we'll need to extend Postgres by installing the following contributed packages: `tablefunc`, `dict_xsyn`, `fuzzystrmatch`, `pg_trgm`, and `cube`. You can refer to the website for installation instructions.[5]

5. http://www.postgresql.org/docs/current/static/contrib.html



Run the following command and check that it matches the output below to ensure your contrib packages have been installed correctly:

```
$ psql 7dbs -c "SELECT '1'::cube;"
cube
------
(1)
(1 row)
```

Seek out the online docs for more information if you receive an error message.

Let's first build the database. It's often good practice to create indexes on foreign keys to speed up reverse lookups (such as what movies this actor is involved in). You should also set a `UNIQUE` constraint on join tables like `movies_actors` to avoid duplicate join values.

postgres/create_movies.sql

```
CREATE TABLE genres (
        name text UNIQUE,
        position integer
);

CREATE TABLE movies (
        movie_id SERIAL PRIMARY KEY,
        title text,
        genre cube
);

CREATE TABLE actors (
        actor_id SERIAL PRIMARY KEY,
        name text
);

CREATE TABLE movies_actors (
        movie_id integer REFERENCES movies NOT NULL,
        actor_id integer REFERENCES actors NOT NULL,
        UNIQUE (movie_id, actor_id)
);
```

```
CREATE INDEX movies_actors_movie_id ON movies_actors (movie_id);
CREATE INDEX movies_actors_actor_id ON movies_actors (actor_id);
CREATE INDEX movies_genres_cube ON movies USING gist (genre);
```

You can download the movies_data.sql file as a file alongside the book and populate the tables by piping the file into the database. Any questions you may have about the genre cube will be covered later today.

Fuzzy Searching

Opening up a system to text searches means opening your system to inaccurate inputs. You have to expect typos like "Brid of Frankstein." Sometimes, users can't remember the full name of "J. Roberts." In other cases, we just plain don't know how to spell "Benn Aflek." We'll look into a few PostgreSQL packages that make text searching easy.

It's worth noting that as we progress, this kind of string matching blurs the lines between relational queries and searching frameworks such as Lucene[6] and Elasticsearch.[7] Although some may feel that features like full-text search belong with the application code, there can be performance and administrative benefits of pushing these packages to the database, where the data lives.

SQL Standard String Matches

PostgreSQL has many ways of performing text matches but the two major default methods are LIKE and regular expressions.

I Like LIKE and ILIKE

LIKE and ILIKE are the simplest forms of text search (ILIKE is a case-insensitive version of LIKE). They are fairly universal in relational databases. LIKE compares column values against a given pattern string. The % and _ characters are wildcards: % matches any number of any characters while _ matches exactly one character.

```
SELECT title FROM movies WHERE title ILIKE 'stardust%';

       title
-------------------
 Stardust
 Stardust Memories
```

If we want to be sure the substring *stardust* is not at the end of the string, we can use the underscore (_) character as a little trick.

6. http://lucene.apache.org/
7. https://www.elastic.co/products/elasticsearch

```
SELECT title FROM movies WHERE title ILIKE 'stardust_%';

       title
-------------------
 Stardust Memories
```

This is useful in basic cases, but LIKE is limited to simple wildcards.

Regex

A more powerful string-matching syntax is a *regular expression* (regex). Regexes appear often throughout this book because many databases support them. There are entire books dedicated to writing powerful expressions—the topic is far too wide and complex to cover in depth here. Postgres conforms (mostly) to the POSIX style.

In Postgres, a regular expression match is led by the ~ operator, with the optional ! (meaning *not* matching) and * (meaning *case insensitive*). To count all movies that do *not* begin with *the*, the following case-insensitive query will work. The characters inside the string are the regular expression.

```
SELECT COUNT(*) FROM movies WHERE title !~* '^the.*';
```

You can index strings for pattern matching the previous queries by creating a text_pattern_ops operator class index, as long as the values are indexed in lowercase.

```
CREATE INDEX movies_title_pattern ON movies (lower(title) text_pattern_ops);
```

We used the text_pattern_ops because the title is of type text. If you need to index varchars, chars, or names, use the related ops: varchar_pattern_ops, bpchar_pattern_ops, and name_pattern_ops.

Bride of Levenshtein

Levenshtein is a string comparison algorithm that compares how similar two strings are by how many *steps* are required to change one string into another. Each replaced, missing, or added character counts as a step. The distance is the total number of steps away. In PostgreSQL, the levenshtein() function is provided by the fuzzystrmatch contrib package. Say we have the string *bat* and the string *fads*.

```
SELECT levenshtein('bat', 'fads');
```

The Levenshtein distance is 3 because in order to go from *bat* to *fads*, we replaced two letters (b=>f, t=>d) and added a letter (+s). Each change increments the distance. We can watch the distance close as we step closer (so to speak). The total goes down until we get zero (the two strings are equal).

```
SELECT levenshtein('bat', 'fad') fad,
  levenshtein('bat', 'fat') fat,
  levenshtein('bat', 'bat') bat;
```

```
 fad | fat | bat
-----+-----+-----
   2 |   1 |   0
```

Changes in case cost a point, too, so you may find it best to convert all strings to the same case when querying.

```
SELECT movie_id, title FROM movies
WHERE levenshtein(lower(title), lower('a hard day nght')) <= 3;
```

```
 movie_id |       title
----------+--------------------
      245 | A Hard Day's Night
```

This ensures minor differences won't over-inflate the distance.

Try a Trigram

A trigram is a group of three consecutive characters taken from a string. The pg_trgm contrib module breaks a string into as many trigrams as it can.

```
SELECT show_trgm('Avatar');
```

```
              show_trgm
-------------------------------------
 {"  a"," av","ar ",ata,ava,tar,vat}
```

Finding a matching string is as simple as counting the number of matching trigrams. The strings with the most matches are the most similar. It's useful for doing a search where you're okay with either slight misspellings or even minor words missing. The longer the string, the more trigrams and the more likely a match—they're great for something like movie titles because they have relatively similar lengths. We'll create a trigram index against movie names to start, using Generalized Index Search Tree (GIST), a generic index API made available by the PostgreSQL engine.

```
CREATE INDEX movies_title_trigram ON movies
USING gist (title gist_trgm_ops);
```

Now you can query with a few misspellings and still get decent results.

```
SELECT title
FROM movies
WHERE title % 'Avatre';
```

```
 title
---------
 Avatar
```

Trigrams are an excellent choice for accepting user input without weighing queries down with wildcard complexity.

Full-Text Fun

Next, we want to allow users to perform full-text searches based on matching words, even if they're pluralized. If a user wants to search for certain words in a movie title but can remember only some of them, Postgres supports simple natural language processing.

TSVector and TSQuery

Let's look for a movie that contains the words *night* and *day*. This is a perfect job for text search using the @@ full-text query operator.

```
SELECT title
FROM movies
WHERE title @@ 'night & day';
```

```
             title
-------------------------------
 A Hard Day's Night
 Six Days Seven Nights
 Long Day's Journey Into Night
```

The query returns titles like *A Hard Day's Night*, despite the word *Day* being in possessive form and the fact that the two words are out of order in the query. The @@ operator converts the name field into a tsvector and converts the query into a tsquery.

A tsvector is a datatype that splits a string into an array (or a *vector*) of tokens, which are searched against the given query, while the tsquery represents a query in some language, like English or French. The language corresponds to a dictionary (which we'll see more of in a few paragraphs). The previous query is equivalent to the following (if your system language is set to English):

```
SELECT title
FROM movies
WHERE to_tsvector(title) @@ to_tsquery('english', 'night & day');
```

You can take a look at how the vector and the query break apart the values by running the conversion functions on the strings outright.

```
SELECT to_tsvector('A Hard Day''s Night'),
  to_tsquery('english', 'night & day');
```

```
      to_tsvector          |   to_tsquery
---------------------------+-----------------
'day':3 'hard':2 'night':5 | 'night' & 'day'
```

The tokens on a tsvector are called *lexemes* and are coupled with their positions in the given phrase.

You may have noticed the tsvector for *A Hard Day's Night* did not contain the lexeme *a*. Moreover, simple English words such as *a* are missing if you try to query by them.

```
SELECT *
FROM movies
WHERE title @@ to_tsquery('english', 'a');
```

```
NOTICE:  text-search query contains only stop words or doesn't \
       contain lexemes, ignored
```

Common words such as *a* are called *stop words* and are generally not useful for performing queries. The English dictionary was used by the parser to normalize our string into useful English components. In your console, you can view the output of the stop words under the English tsearch_data directory.

```
$ cat `pg_config --sharedir`/tsearch_data/english.stop
```

We could remove *a* from the list, or we could use another dictionary like simple that just breaks up strings by nonword characters and makes them lowercase. Compare these two vectors:

```
SELECT to_tsvector('english', 'A Hard Day''s Night');
```

```
      to_tsvector
----------------------------
'day':3 'hard':2 'night':5
```

```
SELECT to_tsvector('simple', 'A Hard Day''s Night');
```

```
            to_tsvector
---------------------------------------
'a':1 'day':3 'hard':2 'night':5 's':4
```

With simple, you can retrieve any movie containing the lexeme *a*.

Other Languages

Because Postgres is doing some natural language processing here, it only makes sense that different configurations would be used for different languages. All of the installed configurations can be viewed with this command:

```
7dbs=# \dF
```

Dictionaries are part of what Postgres uses to generate tsvector lexemes (along with stop words and other tokenizing rules we haven't covered called *parsers* and *templates*). You can view your system's list here:

```
7dbs=# \dFd
```

You can test any dictionary outright by calling the ts_lexize() function. Here we find the English stem word of the string *Day's*.

```
SELECT ts_lexize('english_stem', 'Day''s');
```

```
 ts_lexize
-----------
 {day}
```

Finally, the previous full-text commands work for other languages, too. If you have German installed, try this:

```
SELECT to_tsvector('german', 'was machst du gerade?');
```

```
  to_tsvector
--------------------
'gerad':4 'mach':2
```

Because *was* (what) and *du* (you) are common, they are marked as stop words in the German dictionary, while *machst* (doing) and *gerade* (at the moment) are stemmed.

Indexing Lexemes

Full-text search is powerful. But if we don't index our tables, it's also slow. The EXPLAIN command is a powerful tool for digging into how queries are internally planned.

```
EXPLAIN
SELECT *
FROM movies
WHERE title @@ 'night & day';
```

```
                                QUERY PLAN
-------------------------------------------------------------------------
 Seq Scan on movies  (cost=0.00..815.86 rows=3 width=171)
   Filter: (title @@ 'night & day'::text)
```

Note the line *Seq Scan on movies*. That's rarely a good sign in a query because it means a whole table scan is taking place; each row will be read. That usually means that you need to create an index.

We'll use Generalized Inverted iNdex (GIN)—like GIST, it's an index API—to create an index of lexeme values we can query against. The term *inverted index* may sound familiar to you if you've ever used a search engine like Lucene or Sphinx. It's a common data structure to index full-text searches.

```
CREATE INDEX movies_title_searchable ON movies
USING gin(to_tsvector('english', title));
```

With our index in place, let's try to search again.

```
EXPLAIN
SELECT *
FROM movies
WHERE title @@ 'night & day';
```

```
                                QUERY PLAN
-------------------------------------------------------------------------
 Seq Scan on movies  (cost=0.00..815.86 rows=3 width=171)
   Filter: (title @@ 'night & day'::text)
```

What happened? Nothing. The index is there, but Postgres isn't using it because our GIN index specifically uses the english configuration for building its tsvectors, but we aren't specifying that vector. We need to specify it in the WHERE clause of the query.

```
EXPLAIN
SELECT *
FROM movies
WHERE to_tsvector('english',title) @@ 'night & day';
```

That will return this query plan:

```
                              QUERY PLAN
-----------------------------------------------------------------------
 Bitmap Heap Scan on movies  (cost=20.00..24.26 rows=1 width=171)
   Recheck Cond: (to_tsvector('english'::regconfig, title) @@
   '''night'' & ''day'''::tsquery)
   ->  Bitmap Index Scan on movies_title_searchable
       (cost=0.00..20.00 rows=1 width=0)
       Index Cond: (to_tsvector('english'::regconfig, title) @@
       '''night'' & ''day'''::tsquery)
```

EXPLAIN is important to ensure indexes are used as you expect them. Otherwise, the index is just wasted overhead.

Metaphones

We've inched toward matching less specific inputs. LIKE and regular expressions require crafting patterns that can match strings precisely according to their format. Levenshtein distance allows you to find matches that contain minor misspellings but must ultimately be very close to the same string. Trigrams are a good choice for finding reasonable misspelled matches. Finally, full-text searching allows natural language flexibility in that it can ignore minor words such as *a* and *the* and can deal with pluralization. Sometimes we just don't know how to spell words correctly but we know how they sound.

We love Bruce Willis and would love to see what movies he's in. Unfortunately, we can't remember exactly how to spell his name, so we sound it out as best we can.

```
SELECT *
FROM actors
WHERE name = 'Broos Wils';
```

Even a trigram is no good here (using % rather than =).

```
SELECT *
FROM actors
WHERE name % 'Broos Wils';
```

Enter the metaphones, which are algorithms for creating a string representation of word sounds. You can define how many characters are in the output string. For example, the seven-character metaphone of the name Aaron Eckhart is *ARNKHRT*.

To find all films with actors with names sounding like Broos Wils, we can query against the metaphone output. Note that NATURAL JOIN is an INNER JOIN that automatically joins ON matching column names (for example, movies.actor_id= movies_actors.actor_id).

```
SELECT title
FROM movies NATURAL JOIN movies_actors NATURAL JOIN actors
WHERE metaphone(name, 6) = metaphone('Broos Wils', 6);
```

```
           title
-----------------------------
 The Fifth Element
 Twelve Monkeys
 Armageddon
 Die Hard
 Pulp Fiction
 The Sixth Sense
:
```

If you peek at the online documentation, you'll see the *fuzzystrmatch* module contains other functions: dmetaphone() (double metaphone), dmetaphone_alt() (for alternative name pronunciations), and soundex() (a really old algorithm from the 1880s made by the U.S. Census to compare common American surnames).

You can dissect the functions' representations by selecting their output.

```
SELECT name, dmetaphone(name), dmetaphone_alt(name),
  metaphone(name, 8), soundex(name)
FROM actors;
```

```
      name      | dmetaphone | dmetaphone_alt | metaphone | soundex
----------------+------------+----------------+-----------+--------
 50 Cent        | SNT        | SNT            | SNT       | C530
 Aaron Eckhart  | ARNK       | ARNK           | ARNKHRT   | A652
 Agatha Hurle   | AK0R       | AKTR           | AK0HRL    | A236
```

There is no single best function to choose, and the optimal choice depends on your dataset and use case.

Combining String Matches

With all of our string searching ducks in a row, we're ready to start combining them in interesting ways.

One of the most flexible aspects of metaphones is that their outputs are just strings. This allows you to mix and match with other string matchers.

For example, we could use the trigram operator against metaphone() outputs and then order the results by the lowest Levenshtein distance. This means "Get me names that sound the most like Robin Williams, in order."

```
SELECT * FROM actors
WHERE metaphone(name,8) % metaphone('Robin Williams',8)
ORDER BY levenshtein(lower('Robin Williams'), lower(name));
```

```
 actor_id | name
----------+-----------------
     4093 | Robin Williams
     2442 | John Williams
     4479 | Steven Williams
     4090 | Robin Shou
```

Be warned, though, that unbridled exploitation of this flexibility can yield funny results.

```
SELECT * FROM actors WHERE dmetaphone(name) % dmetaphone('Ron');
```

This will return a result set that includes actors like Renée Zellweger, Ringo Starr, and Randy Quaid.

The combinations are vast, limited only by your experimentations.

Genres as a Multidimensional Hypercube

The last contributed package we investigate is cube. We'll use the cube datatype to map a movie's genres as a multidimensional vector. We will then use methods to efficiently query for the closest points within the boundary of a hypercube to give us a list of similar movies.

As you may have noticed in the beginning of Day 3, we created a column named genres of type cube. Each value is a point in 18-dimensional space with each dimension representing a genre. Why represent movie genres as points in n-dimensional space? Movie categorization is not an exact science, and many movies are not 100 percent comedy or 100 percent tragedy—they are something in between.

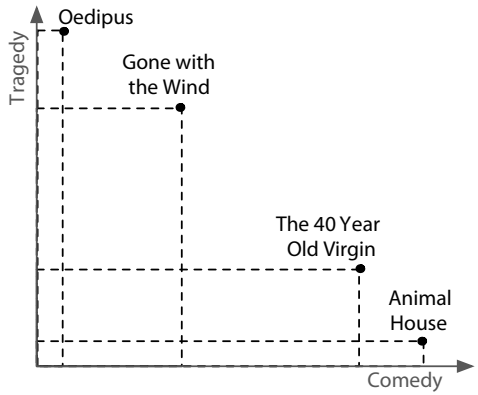
In our system, each genre is scored from (the totally arbitrary numbers) 0 to 10 based on how strong the movie is within that genre—with 0 being nonexistent and 10 being the strongest.

Star Wars, for example, has a genre vector of (0,7,0,0,0,0,0,0,0,7,0,0,0,0,10,0,0,0). The genres table describes the position of each dimension in the vector. We can decrypt its genre values by extracting the cube_ur_coord(vector,dimension) using each genres.position. For clarity, we filter out genres with scores of 0.

```
SELECT name,
  cube_ur_coord('(0,7,0,0,0,0,0,0,0,7,0,0,0,0,10,0,0,0)', position) as score
FROM genres g
WHERE cube_ur_coord('(0,7,0,0,0,0,0,0,0,7,0,0,0,0,10,0,0,0)', position) > 0;
```

```
   name     | score
------------+-------
 Adventure |     7
 Fantasy   |     7
 SciFi     |    10
```

We will find similar movies by finding the nearest points. To understand why this works, we can envision two movies on a two-dimensional genre graph, like the graph shown below. If your favorite movie is *Animal House*, you'll probably want to see *The 40-Year-Old Virgin* more than *Oedipus*—a story famously lacking in comedy. In our two-dimensional universe, it's a simple nearest-neighbor search to find likely matches.



We can extrapolate this into more dimensions with more genres, be it 2, 3, or 18. The principle is the same: a nearest-neighbor match to the nearest points in genre space will yield the closest genre matches.

The nearest matches to the genre vector can be discovered by the cube_distance(point1, point2). Here we can find the distance of all movies to the *Star Wars* genre vector, nearest first.

```
SELECT *,
  cube_distance(genre, '(0,7,0,0,0,0,0,0,0,7,0,0,0,0,10,0,0,0)') dist
FROM movies
ORDER BY dist;
```

We created the movies_genres_cube cube index earlier when we created the tables. However, even with an index, this query is still relatively slow because it requires a full-table scan. It computes the distance on every row and then sorts them.

Rather than compute the distance of every point, we can instead focus on likely points by way of a *bounding cube*. Just like finding the closest five towns on a map will be faster on a state map than a world map, bounding reduces the points we need to look at.

We use cube_enlarge(cube,radius,dimensions) to build an 18-dimensional cube that is some length (radius) wider than a point.

Let's view a simpler example. If we built a two-dimensional square one unit around a point (1,1), the lower-left point of the square would be at (0,0), and the upper-right point would be (2,2).

```
SELECT cube_enlarge('(1,1)',1,2);

 cube_enlarge
---------------
 (0, 0),(2, 2)
```

The same principle applies in any number of dimensions. With our bounding hypercube, we can use a special cube operator, @>, which means *contains*. This query finds the distance of all points contained within a five-unit cube of the *Star Wars* genre point.

```
SELECT title,
  cube_distance(genre, '(0,7,0,0,0,0,0,0,0,7,0,0,0,0,10,0,0,0)') dist
FROM movies
WHERE cube_enlarge('(0,7,0,0,0,0,0,0,0,7,0,0,0,0,10,0,0,0)'::cube, 5, 18)
  @> genre
ORDER BY dist;
```

```
                    title                     |       dist
----------------------------------------------+------------------
 Star Wars                                    |                0
 Star Wars: Episode V - The Empire Strikes Back |                2
 Avatar                                       |                5
 Explorers                                    | 5.74456264653803
 Krull                                        | 6.48074069840786
 E.T. The Extra-Terrestrial                   | 7.61577310586391
```

Using a subselect, we can get the genre by movie name and perform our calculations against that genre using a table alias.

```
SELECT m.movie_id, m.title
FROM movies m, (SELECT genre, title FROM movies WHERE title = 'Mad Max') s
WHERE cube_enlarge(s.genre, 5, 18) @> m.genre AND s.title <> m.title
ORDER BY cube_distance(m.genre, s.genre)
LIMIT 10;
```

```
 movie_id |           title
----------+---------------------------
     1405 | Cyborg
     1391 | Escape from L.A.
     1192 | Mad Max Beyond Thunderdome
     1189 | Universal Soldier
     1222 | Soldier
     1362 | Johnny Mnemonic
      946 | Alive
      418 | Escape from New York
     1877 | The Last Starfighter
     1445 | The Rocketeer
```

This method of movie suggestion is not perfect, but it's an excellent start. We will see more dimensional queries in later chapters, such as two-dimensional geographic searches in MongoDB (see *GeoSpatial Queries*, on page 130).

Day 3 Wrap-Up

Today we jumped headlong into PostgreSQL's flexibility in performing string searches and used the `cube` package for multidimensional searching. Most importantly, we caught a glimpse of the nonstandard extensions that put PostgreSQL at the top of the open source RDBMS field. There are dozens (if not hundreds) more extensions at your disposal, from geographic storage to cryptographic functions, custom datatypes, and language extensions. Beyond the core power of SQL, contrib packages make PostgreSQL shine.

Day 3 Homework

Find

1. Find the online documentation listing all contributed packages bundled into Postgres. Read up on two that you could imagine yourself using in one of your projects.

2. Find the online POSIX regex documentation (it will also come in handy in future chapters).

Do

1. Create a stored procedure that enables you to input a movie title or an actor's name and then receive the top five suggestions based on either movies the actor has starred in or films with similar genres.

2. Expand the movies database to track user comments and extract keywords (minus English stopwords). Cross-reference these keywords with actors' last names and try to find the most talked-about actors.

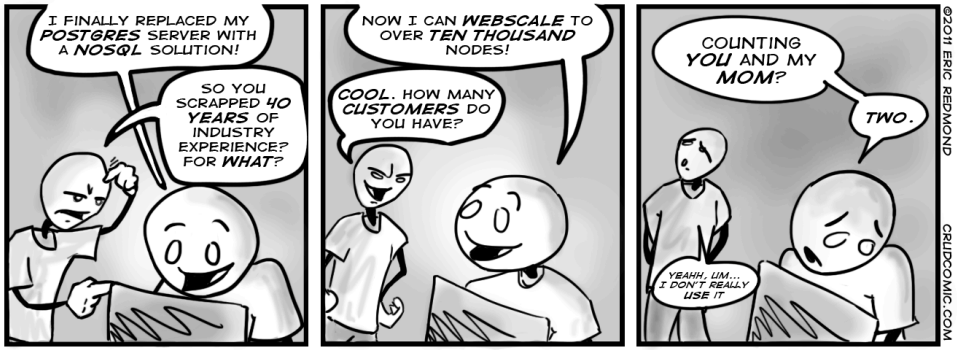
Wrap-Up

If you haven't spent much time with relational databases, we highly recommend digging deeper into PostgreSQL, or another relational database, before deciding to scrap it for a newer variety. Relational databases have been the focus of intense academic research and industrial improvements for more than forty years, and PostgreSQL is one of the top open source relational databases to benefit from these advancements.

PostgreSQL's Strengths

PostgreSQL's strengths are as numerous as any relational model: years of research and production use across nearly every field of computing, flexible queryability, and very consistent and durable data. Most programming languages have battle-tested driver support for Postgres, and many programming models, like object-relational mapping (ORM), assume an underlying relational database.

But the real crux of the matter is the flexibility of the join. You don't need to know how you plan to actually query your model because you can always perform some joins, filters, views, and indexes—odds are good that you will always have the ability to extract the data you want. In the other chapters of this book that assumption will more or less fly out the window.



PostgreSQL is fantastic for what we call "Stepford data" (named for *The Stepford Wives*, a story about a neighborhood where nearly everyone was consistent in style and substance), which is data that is fairly homogeneous and conforms well to a structured schema.

Furthermore, PostgreSQL goes beyond the normal open source RDBMS offerings, such as powerful schema constraint mechanisms. You can write your own language extensions, customize indexes, create custom datatypes, and even overwrite the parsing of incoming queries. And where other open source databases may have complex licensing agreements, PostgreSQL is open source in its purest form. No one owns the code. Anyone can do pretty much anything they want with the project (other than hold authors liable). The development and distribution are completely community supported. If you are a fan of free(dom) software, you have to respect their general resistance to cashing in on an amazing product.

PostgreSQL's Weaknesses

Although relational databases have been undeniably the most successful style of database over the years, there are cases where it may not be a great fit.

Postgres and JSON

Although we won't delve too deeply into it here given that we cover two other databases in this book that were explicitly created to handle unstructured data, we'd be remiss in not mentioning that Postgres has offered support for JSON since version 9.3. Postgres offers two different formats for this: JSON and JSONB (the `json` and `jsonb` types, respectively). The crucial difference between them is that the `json` type stores JSON as text while `jsonb` stores JSON using a decomposed binary format; `json` is optimized for faster data input while `jsonb` is optimized for faster processing.

With Postgres, you can perform operations like this:

```
CREATE TABLE users (
  username TEXT,
  data     JSON
);
INSERT INTO users VALUES ('wadeboggs107', '{ "AVG": 0.328, "HR": 118, "H": 3010 }');
SELECT data->>'AVG' AS lifetime_batting_average FROM users;

 lifetime_batting_average
--------------------------
 0.328
```

If your use case requires a mixture of structured and unstructured (or less structured) datatypes—or even requires *only* unstructured datatypes—then Postgres may provide a solution.

Partitioning is not one of the strengths of relational databases such as PostgreSQL. If you need to scale out rather than up—multiple parallel databases rather than a single beefy machine or cluster—you may be better served looking elsewhere (although clustering capabilities have improved in recent releases). Another database might be a better fit if:

- You don't truly require the overhead of a full database (perhaps you only need a cache like Redis).
- You require very high-volume reads and writes as key values.
- You need to store only large BLOBs of data.

Parting Thoughts

A relational database is an excellent choice for query flexibility. While PostgreSQL requires you to design your data up front, it makes no assumptions about how you use that data. As long as your schema is designed in a fairly normalized way, without duplication or storage of computable values, you should generally be all set for any queries you might need to create. And if you include the correct modules, tune your engine, and index well, it will perform amazingly well for multiple terabytes of data with very small resource consumption. Finally, to those for whom data safety is paramount, PostgreSQL's ACID-compliant transactions ensure your commits are completely atomic, consistent, isolated, and durable.